Guia de Estudo – Arduino e Hardware (SuperSenha, Projeto 8)

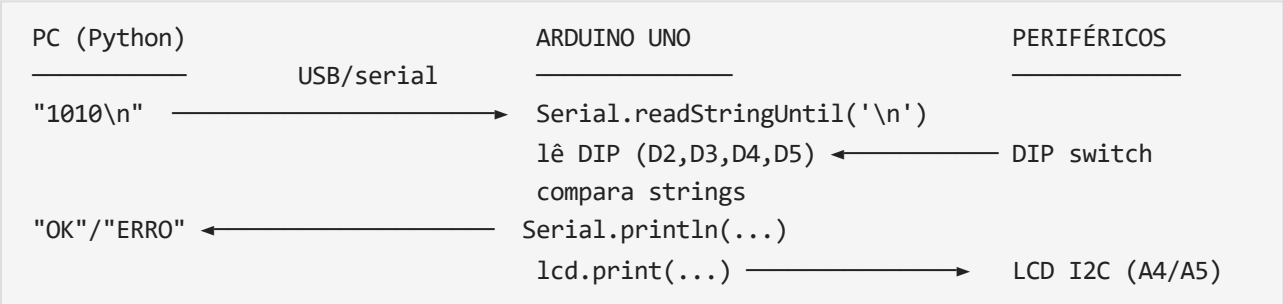
Arquivos estudados: `supersenha_arduino.ino`, `scanner_i2c.ino` e o *Guia de Montagem Física*. Cobre as duas áreas que o professor pode te dar: **Software Arduino** (Arduino IDE) e **Hardware** (montagem do circuito).

Lembrete da nota: 1ª pergunta certa = 9, 2ª = 7, 3ª = 5, 4ª = 3. Domine o Nível 1 primeiro.

---

1. Resumo em 30 segundos (decore)

> "O Arduino UNO recebe pela serial USB a senha digitada no Python (ex.: `1010`). Ele lê as 4 chaves do DIP switch nos pinos D2 a D5, monta a senha real em texto e compara com a recebida. Se for igual, escreve **ACESSO LIBERADO!** no LCD 16x2 I2C e manda `OK` para o Python. Se for diferente, escreve **SENHA ERRADA!** e manda `ERRO`. As chaves usam `INPUT_PULLUP`, então chave ON lê LOW, que o código interpreta como '1'. O LCD conversa pelo barramento I2C (SDA no A4, SCL no A5) no endereço `0x27`."



---

2. Hardware: o que é cada componente

Componente	Função no projeto
Arduino UNO	Microcontrolador (ATmega328P, 5 V, 16 MHz). Lê as chaves, compara e comanda o LCD
Cabo USB	Alimenta o Arduino **e** leva a comunicação serial com o PC (porta COM)
DIP switch de 4 chaves	4 interruptores. Define a senha física. ON = 1, OFF = 0
Display LCD 16x2	Mostra texto: 16 colunas × 2 linhas (controlador HD44780)
Módulo I2C (backpack)	Plaquinha soldada atrás do LCD que reduz ~16 fios para **4** (VCC, GND, SDA, SCL). Contém o chip conversor PCF8574 e o trimpot de contraste
Protoboard	Placa de testes sem solda. As trilhas laterais (+ e −) distribuem 5 V e GND
Jumpers macho-fêmea	Ligam os pinos do módulo I2C (que são pinos macho) na protoboard/Arduino

Componente	Função no projeto
Jumpers macho-macho	Ligam DIP, 5 V e GND
Trimpot azul	Ajusta o contraste do LCD (parafusinho)

Esquema de ligação (decore a tabela)

DIP switch	Arduino		Módulo I2C	Liga em
Chave 1	D2		VCC	5V
Chave 2	D3		GND	GND
Chave 3	D4		SDA	**A4**
Chave 4	D5		SCL	**A5**
Outro lado das 4 chaves	GND			

Preparação da protoboard: 5V do Arduino na trilha vermelha (+) e GND na trilha azul (−).

O DIP deve ficar **atravessando o canal do meio** da protoboard. Assim os dois pinos de cada chave ficam em lados separados, sem curto entre eles.

Perguntas de Hardware

H1. Por que não há resistor nas chaves do DIP? Porque o código usa `INPUT_PULLUP`: o Arduino liga um resistor interno (aprox. 20 a 50 kΩ) entre o pino e o 5 V. Não precisa resistor externo.

H2. Por que o outro lado das chaves vai no GND? Com a chave ON, o pino fica ligado ao GND e lê LOW (0 V). Com a chave OFF, o pino "flutua" e o pull-up interno o puxa para HIGH (5 V). É o que gera dois estados bem definidos.

H3. Por que o LCD usa só 4 fios? Por causa do módulo I2C. Um LCD 16x2 sozinho precisa de uns 6 a 10 pinos de dados/controle. O I2C usa só 2 fios de sinal (SDA e SCL) mais alimentação.

H4. Por que SDA no A4 e SCL no A5? No Arduino UNO, o hardware I2C fixo está nesses pinos: **A4 = SDA** e **A5 = SCL**. Não dá para trocar por outros.

H5. Se os fios SDA e SCL forem trocados, o que acontece? O LCD não é encontrado (o scanner não acha endereço) e não aparece texto. Não queima, mas não funciona.

H6. O LCD acende mas não aparece texto (ou só quadrados). E agora? Girar o trimpot azul de contraste com a chave de fenda. Se mesmo assim nada, rodar o `scanner_i2c.ino` para conferir o endereço.

H7. LCD totalmente apagado? Conferir 5V/GND e o jumper de backlight do módulo.

H8. Arduino reinicia ou desliga sozinho? Possível curto entre 5V e GND. Desconectar o USB e revisar os fios.

H9. Por que montar com o USB desconectado? Para evitar curto ou ligação errada energizada, que pode queimar componentes ou a porta USB. Só se liga depois do checklist.

H10. Qual o checklist antes de ligar? 5V/GND nas trilhas certas; LCD VCC→5V e GND→GND; SDA→A4 e SCL→A5; DIP 1→D2, 2→D3, 3→D4, 4→D5; outro lado do DIP no GND; nenhum fio solto encostando em outro.

H11. Por que o Arduino pode ser alimentado e comunicar pelo mesmo cabo USB? O USB entrega 5 V para alimentação e também tem os fios de dados, que o chip conversor USB-serial do Arduino transforma em comunicação serial (pinos 0 e 1 internamente).

H12. Senha sempre errada. O que conferir? A ordem das chaves (chave 1 em D2...) e se o outro lado das chaves está de fato no GND. Se a ordem estiver trocada, a senha do DIP sai invertida.

---

3. Software: código `supersenha_arduino.ino` explicado linha a linha

Bibliotecas e definições (linhas 1–10)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#define pinoChave1 2   ... #define pinoChave4 5
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

- `#include <Wire.h>`: biblioteca **nativa** do Arduino para I2C.
- `#include <LiquidCrystal_I2C.h>`: biblioteca **externa** (instalar "LiquidCrystal I2C", de Frank de Brabander). Dá os comandos `init`, `backlight`, `setCursor`, `print`, `clear`.
- `#define`: dá **nomes** aos números dos pinos. O pré-processador troca `pinoChave1` por `2` antes de compilar. Facilita ler e alterar o código.
- `LiquidCrystal_I2C lcd(0x27, 16, 2)`: cria o **objeto** `lcd`. Parâmetros: endereço I2C (`0x27`), 16 colunas, 2 linhas. Se o seu módulo for diferente, o endereço pode ser `0x3F`.

`setup()` (linhas 12–30): roda **uma vez** ao ligar/reiniciar

Comando	O que faz
`Serial.begin(9600)`	Liga a serial a 9600 bits/s (igual ao Python)
`pinMode(pino, INPUT_PULLUP)`	Pino como entrada com resistor pull-up interno
`lcd.init()`	Inicializa o LCD
`lcd.backlight()`	Liga a luz de fundo
`lcd.clear()`	Apaga a tela
`lcd.setCursor(0, 0)`	Posiciona o cursor: **coluna 0, linha 0** (a contagem começa em 0)
`lcd.print("...")`	Escreve texto na posição do cursor

Tela inicial: linha 0 `" SuperSenha "`, linha 1 `"Aguardando... "`. Os espaços completam as **16 colunas** e apagam sobras de textos antigos.

`loop()` (linhas 32–58): repete **para sempre**

1. `if (Serial.available() > 0)`: só age se chegou algo pela serial.
2. `Serial.readStringUntil('\n')`: lê os caracteres até o `\n` (é por isso que o Python manda `"1010\n"`).
3. `senhaDigitada.trim()`: tira espaços e `\r` / `\n` sobrando nas pontas.

4. Monta `senhaDIP`: para cada chave, se `digitalRead(pino) == LOW` acrescenta `"1"`, senão `"0"`.
5. `lcd.clear()`: limpa a tela para o novo resultado.
6. Se `senhaDigitada == senhaDIP`: escreve **ACESSO LIBERADO!** e envia `Serial.println("OK")`.
7. Senão: escreve **SENHA ERRADA!** e envia `Serial.println("ERRO")`.

A lógica invertida do pull-up (a pegadinha mais provável!)

Posição da chave	Ligação do pino	`digitalRead`	Dígito da senha
ON	Conectado ao GND	`LOW`	**1**
OFF	Solto (pull-up puxa p/ 5 V)	`HIGH`	**0**

O sinal elétrico é o contrário da lógica da senha (**lógica invertida / ativo em baixo**). O código "desinverte" com `== LOW` → `"1"`.

Exemplo: chaves ON-OFF-ON-OFF → leituras LOW-HIGH-LOW-HIGH → senha `"1010"`.

O programa de apoio `scanner_i2c.ino`

Serve para **descobrir o endereço I2C** do LCD quando o texto não aparece.

- `Wire.begin()`: inicia o I2C como mestre.
- O `for` testa os endereços de 1 a 126: `Wire.beginTransmission(endereco)` e `Wire.endTransmission()`.
- `endTransmission()` devolve **0** quando algum dispositivo respondeu (ACK) naquele endereço, e o endereço é impresso em hexadecimal (`HEX`).
- Se `endereco < 16`, imprime um `"0"` antes para ficar com 2 dígitos (ex.: `0x08`).
- Se não achar nada, avisa para conferir SDA/SCL, VCC e GND.
- `delay(3000)`: repete a varredura a cada 3 s. Ver o resultado no **Monitor Serial a 9600 baud**.

---

4. Perguntas prováveis de Software Arduino, com resposta

NÍVEL 1 – Básicas (9 pontos)

1. O que o código do Arduino faz? Recebe a senha do Python pela serial, lê o DIP switch, compara as duas, mostra o resultado no LCD e responde `OK` ou `ERRO` ao Python.

2. O que são `setup()` e `loop()`? `setup()` roda **uma vez** quando o Arduino liga ou reinicia (configurações). `loop()` roda **repetidamente** para sempre (lógica principal).

3. O que faz `Serial.begin(9600)`? Abre a comunicação serial a 9600 bits por segundo. Precisa ser igual ao `baudrate` do Python.

4. O que faz `pinMode(pino, INPUT_PULLUP)`? Define o pino como entrada e ativa o resistor de pull-up interno. Sem chave pressionada ele lê HIGH.

5. O que faz `digitalRead()`? Lê o nível lógico do pino: `HIGH` (5 V) ou `LOW` (0 V).

6. Por que a chave ON vira "1" se ela lê LOW? Por causa do pull-up: ON liga o pino ao GND, então lê LOW. O programador escolheu tratar "chave ligada" como `1`, então `LOW` vira `"1"`.

7. Que biblioteca controla o LCD e qual a diferença para a `Wire`? `LiquidCrystal_I2C` controla o LCD (comandos de texto). `Wire` é a base do protocolo I2C. A primeira usa a segunda por baixo.

8. O que é o endereço `0x27`? É o endereço I2C do módulo do LCD, em hexadecimal. Cada dispositivo no barramento tem um endereço único. Alguns módulos usam `0x3F`.

9. O que `lcd.setCursor(0, 1)` faz? Coloca o cursor na coluna 0 da **segunda linha** (linhas e colunas começam em 0).

10. Como o Arduino responde ao Python? Com `Serial.println("OK")` ou `Serial.println("ERRO")`.

NÍVEL 2 – Intermediárias (7 pontos)

11. Para que serve `Serial.available()`? Retorna quantos bytes chegaram e estão no buffer. Se for 0, não há nada para ler, então o `loop` não faz nada.

12. O que faz `readStringUntil('\n')`? Lê os caracteres recebidos até encontrar o `\n` e devolve uma `String` (sem o `\n`). Tem um timeout padrão de 1 segundo caso o `\n` nunca chegue.

13. Por que o `.trim()`? Para remover `\r`, `\n` e espaços das pontas. Assim `"1010\r"` não fica diferente de `"1010"` na comparação.

14. Por que a senha do DIP é montada como `String` e não número? Para comparar direto com o texto recebido do Python (`"1010" == "1010"`). Se fossem números ou bits, teríamos que converter.

15. Por que usar `==` entre `String`s funciona aqui? A classe `String` do Arduino sobrecarrega `==`: compara o **conteúdo**. (Em C puro, comparar `char*` com `==` compararia endereços de memória.)

16. Explique `lcd.clear()` antes de escrever o resultado. Apaga o texto anterior ("Aguardando..." ou resultado antigo) para não sobrarem caracteres misturados.

17. Por que as mensagens do LCD têm espaços (`" SENHA ERRADA! "`)? Para ter exatamente 16 caracteres, centralizar o texto e cobrir a linha inteira.

18. O que é `#define pinoChave1 2`? Uma constante do pré-processador. Antes de compilar, todo `pinoChave1` vira `2`. Facilita manutenção: se mudar o pino, edita-se só um lugar.

19. Qual a diferença entre `Serial.print` e `Serial.println`? O `println` acrescenta ao final `\r\n` (retorno + nova linha). O Python usa `readline()`, que **precisa** desse `\n` para terminar a leitura, então tem que ser `println`.

20. Por que o Arduino só lê o DIP quando chega uma senha? A leitura está dentro do `if (Serial.available() > 0)`. Isso é suficiente porque só precisamos do estado do DIP **no momento da tentativa**.

21. O que significa `#include <Wire.h>`? Inclui a biblioteca I2C do Arduino. O display precisa dela para comunicar em A4/A5.

NÍVEL 3 – Mais profundas (5 pontos)

22. O que é o protocolo I2C? Barramento serial síncrono com **2 fios**: **SDA** (dados) e **SCL** (clock). Tem um **mestre** (Arduino) e **escravos** (LCD), cada um com endereço de 7 bits. Permite várias peças nos mesmos 2 fios.

23. Qual a diferença entre comunicação serial UART (USB) e I2C? UART é **assíncrona** (sem clock, depende do baud rate igual dos dois lados), ponto a ponto, 2 fios (TX/RX). I2C é **síncrona** (tem fio de clock), com endereços, vários dispositivos num barramento. No projeto: UART = PC↔Arduino; I2C = Arduino↔LCD.

24. O que é o pull-up e por que ele é necessário? Um resistor ligado ao 5 V que garante nível HIGH quando a entrada não está sendo puxada para baixo. Sem ele, o pino "flutua" e lê valores aleatórios por ruído.

25. O que acontece se usar `INPUT` em vez de `INPUT_PULLUP` sem resistor externo? O pino fica flutuando quando a chave está OFF: leituras instáveis, senha aleatória.

26. Por que o Arduino reinicia quando o Python abre a porta? Ao abrir a serial, o PC ativa o sinal **DTR**, que provoca o reset do UNO (auto-reset do bootloader). Por isso o Python espera 2,5 s e o `setup()` roda de novo, mostrando "Aguardando..." de novo no LCD.

27. Como o Python e o Arduino sabem que estão falando a mesma coisa? Combinaram um **protocolo**: velocidade 9600; senha como 4 caracteres `0/1` terminados em `\n`; resposta `OK` ou `ERRO` terminada em `\n`.

28. Por que o código usa o tipo `String` e qual seu risco? É prático, mas usa memória dinâmica e, em programas grandes de longa duração, pode fragmentar a RAM (o UNO tem só 2 KB). Aqui o programa é pequeno, então não causa problema. A alternativa mais segura é vetor de `char`.

29. O que o `scanner_i2c` retorna em `Wire.endTransmission()`? `0` = sucesso (dispositivo respondeu). Outros valores indicam erro (ex.: 2 = NACK ao enviar endereço, ou seja, ninguém naquele endereço).

30. Como achar o endereço do LCD se `0x27` não funcionar? Carregar o `scanner_i2c.ino`, abrir o Monitor Serial em 9600 e ler o endereço impresso (por ex. `0x3F`). Depois corrigir em `LiquidCrystal_I2C lcd(0x3F, 16, 2);`.

31. Como funciona o fluxo em `loop()` de forma resumida? Espera mensagem → lê senha → lê DIP → compara → mostra no LCD → responde ao Python → volta a esperar.

NÍVEL 4 – Pegadinhas / "e se..." (3 pontos)

32. E se mudar a chave do DIP depois de enviar a senha? A tela muda sozinha? Não. O DIP só é lido no momento em que chega a senha. O resultado fica no LCD até a próxima tentativa.

33. O LCD volta a mostrar "Aguardando..." depois do resultado? Não. O código não faz isso: o resultado fica até a próxima tentativa (ou até o Arduino reiniciar). Seria uma melhoria: `delay(2000)` e reescrever "Aguardando...".

34. E se o Python enviar algo diferente de 4 dígitos, como `"abc"`? O Arduino não valida formato: só compara. Como `"abc"` ≠ a senha do DIP, responde `ERRO`. A validação principal está no Python.

35. E se duas senhas forem enviadas rápido demais? O buffer serial guarda os bytes, mas o `readStringUntil` pega uma por vez. Como o `loop` é rápido e o usuário clica manualmente, não é problema na prática.

36. E se o Monitor Serial do Arduino IDE estiver aberto com o Python rodando? O Python não consegue abrir a porta (acesso negado). Só um programa usa a COM por vez. Fechar o Monitor Serial.

37. E se o baud rate do Arduino fosse 115200 e o do Python 9600? Os bytes chegariam como caracteres corrompidos; a senha nunca seria igual e sempre daria "SENHA ERRADA" (ou nem responderia).

38. Posso usar os pinos A4/A5 para o DIP e outros para o LCD? Para o I2C **não**: A4/A5 são fixos no UNO. O DIP pode ir em quaisquer pinos digitais livres, mudando os `#define`.

39. Se eu quisesse 8 chaves, o que mudaria? Mais pinos/`#define`, mais linhas de `pinMode` e `digitalRead`, e a senha teria 8 dígitos. Um `for` com vetor de pinos deixaria o código mais curto.

40. Dá para evitar repetir os 4 `if`? Sim, com vetor e laço:

```
int pinos[4] = {2, 3, 4, 5};
for (int i = 0; i < 4; i++) {
  senhaDIP += (digitalRead(pinos[i]) == LOW) ? "1" : "0";
}
```

41. Qual a tensão de trabalho de tudo isso? 5 V (Arduino UNO, LCD e módulo I2C ligados no pino 5V). Lógica TTL 5 V: HIGH ≈ 5 V, LOW ≈ 0 V.

42. Quem ocupa os pinos 0 e 1 do UNO? A serial (RX e TX), usada pelo USB. Por isso o DIP começa no pino 2.

---

5. Exercícios de "mexer no código" (o professor pode pedir ao vivo)

1. **Trocar o endereço do LCD**: `lcd(0x3F, 16, 2)` (linha 10).
2. **Mudar as mensagens do LCD**: texto das linhas 50 e 54 (cuidado: 16 caracteres).
3. **Mudar o pino de uma chave**: alterar o `#define` correspondente (linhas 4–7).
4. **Inverter a lógica**: trocar `== LOW` por `== HIGH` (chave ON passaria a ser 0).
5. **Fazer o LCD voltar a "Aguardando..." depois de 2 s**: após o `if/else`, `delay(2000);` e reescrever as duas linhas iniciais.
6. **Acender o LED do pino 13 quando acertar**: `pinMode(13, OUTPUT);` no `setup` e `digitalWrite(13, HIGH);` no bloco do `OK`.

---

6. Cola rápida (revise 5 minutos antes)

Item	Valor
Baud rate	9600
DIP	D2, D3, D4, D5 + GND
LCD I2C	5V, GND, SDA = **A4**, SCL = **A5**

Item	Valor
Endereço LCD	`0x27` (ou `0x3F`)
LCD	16 colunas × 2 linhas
Chave ON	LOW → "1"
Chave OFF	HIGH → "0"
Python → Arduino	`"1010\n"`
Arduino → Python	`"OK\n"` ou `"ERRO\n"`
Reset ao abrir serial	Sim (DTR), por isso `sleep(2.5)` no Python
Contraste fraco	Girar o trimpot azul
Descobrir endereço	`scanner_i2c.ino` + Monitor Serial 9600

7. Dicas para a apresentação

- Aponte no código cada requisito do enunciado: 4 chaves DIP → `pinoChave1..4` e `INPUT_PULLUP`; LCD I2C informa acesso → `lcd.print` nos dois ramos do `if`; comunicação com a interface → `Serial.readStringUntil` e `Serial.println`.
- Saiba **desenhar de cabeça** a ligação: DIP em D2–D5 + GND; LCD em 5V, GND, A4, A5.
- Antes de apresentar: Monitor Serial **fechado**, porta COM correta no Python e contraste do LCD ajustado.
- Se não souber algo, explique a lógica que entendeu em voz alta. O professor valoriza o raciocínio.